# QuestLog

*Znajdź. Oceń. Skomentuj.*

Lekki serwis społecznościowy typu link-aggregator
z głosowaniem na treści i drzewiastymi komentarzami.

## 01 Czym jest QuestLog

QuestLog to lekki serwis społecznościowy typu link-aggregator, zbudowany wokół trzech prostych mechanizmów: zgłaszania znalezisk, głosowania na nie oraz prowadzenia dyskusji w komentarzach. Projekt łączy w sobie najważniejsze elementy aplikacji webowej z kontem użytkownika — od rejestracji, przez tablicę treści, po system ocen i wielopoziomową dyskusję.

### Trzy filary aplikacji

- Tablica znalezisk. Użytkownicy wrzucają ciekawe linki i krótkie teksty, które trafiają na wspólną listę.
- Głosy decydują. Społeczność ocenia treści głosami w górę i w dół, wynosząc najlepsze wpisy na szczyt.
- Dyskusja pod każdym wpisem. Komentarze mają formę drzewka — podobnie jak na Reddicie — z nieograniczoną liczbą poziomów odpowiedzi.

| 3 | 4 | ∞ |
|---|---|---|
| warstwy architektury: React, Flask, PostgreSQL | tabele w bazie danych: użytkownicy, wpisy, komentarze, głosy | poziomów zagnieżdżenia odpowiedzi w komentarzach |

## 02 Jak to działa

Ścieżka użytkownika w QuestLog jest krótka i intuicyjna — od założenia konta, przez przeglądanie i dodawanie treści, po głosowanie i udział w dyskusji.

**01** **Konto**

Użytkownik zakłada konto i loguje się — akcje takie jak dodawanie wpisów, głosowanie czy komentowanie wymagają zalogowania.

**02** **Tablica**

Przegląda tablicę znalezisk, sortowaną według: gorące, najnowsze albo najlepsze.

**03** **Dodawanie**

Dodaje nowy wpis — link lub krótki tekst, opcjonalnie z tagiem tematycznym.

**04** **Głosowanie**

Oddaje głos: upvote albo downvote, a wynik aktualizuje się natychmiast.

**05** **Komentarze**

Dołącza do dyskusji — odpowiedzi układają się w drzewko pod każdym znaleziskiem.

## 03 Funkcjonalności

Zakres funkcjonalny aplikacji obejmuje sześć głównych obszarów, odpowiadających kolejnym etapom pracy z aplikacją — od zarządzania kontem po moderację własnych treści.

| | |
|---|---|
| **Konto** | Rejestracja i logowanie; akcje takie jak dodawanie, głosowanie i komentowanie dostępne tylko po zalogowaniu. |
| **Tablica** | Sortowanie wpisów: gorące, najnowsze, najlepsze. |
| **Nowy wpis** | Tytuł, link lub krótki tekst, opcjonalnie tag tematyczny. |
| **Głosowanie** | Upvote i downvote zarówno na wpisy, jak i na komentarze. |
| **Komentarze** | Drzewiaste odpowiedzi z opcją przełączenia na widok płaski. |
| **Moje treści** | Edycja i usuwanie własnych wpisów oraz komentarzy. |

## 04 Komentarze drzewiaste

Jednym z ciekawszych wyzwań projektu jest zbudowanie struktury komentarzy w formie drzewa, w której każda odpowiedź może mieć kolejne odpowiedzi — bez ograniczenia głębokości zagnieżdżenia.

| Znalezisko | |
|---|---|
| Komentarz A | parent_id: brak |
| Odpowiedź A1 | parent_id: A |
| Odpowiedź A1a | parent_id: A1 |
| Komentarz B | parent_id: brak |

**Jak to wygląda w bazie**

- Każdy komentarz ma pole parent_id.
- Puste pole oznacza komentarz główny.
- Wypełnione — wskazuje komentarz nadrzędny.
- Odpowiedzi mogą zagnieżdżać się bez limitu.

## 05 Technologie

Aplikacja opiera się na trójwarstwowej architekturze, łączącej nowoczesny front-end, lekki back-end oraz relacyjną bazę danych.

| **React**<br>Front-end | **Python + Flask**<br>Back-end | **PostgreSQL**<br>Baza w kontenerze Docker |
|---|---|---|
| Komponenty: tablica, wpis, drzewo komentarzy, formularze. | Prosty REST API i logika głosowania. | Jedno polecenie na start, zapytania w zwykłym SQL. |

## 06 Kod na GitHubie

Cały kod projektu jest wersjonowany i przechowywany w repozytorium GitHub, co zapewnia pełną historię zmian, możliwość pracy zespołowej oraz bezpieczną kopię zapasową.

- Historia zmian. Można wrócić do każdej wcześniejszej wersji kodu.
- Praca w zespole. Osobne branche i pull requesty dla poszczególnych funkcjonalności.
- Kopia zapasowa. Kod jest bezpiecznie przechowywany w chmurze.
- Do portfolio. Projekt jest widoczny na profilu i może być prezentowany dalej.

## 07 Model danych

Baza danych składa się z czterech powiązanych tabel, odzwierciedlających główne obiekty aplikacji: użytkowników, wpisy, komentarze i głosy.

| User | Post | Comment | Vote |
|---|---|---|---|
| id | id | id | id |
| username | title | post_id | user_id |
| email | url | author_id | post_id lub comment_id |
| password_hash | body | parent_id | value (+1 / −1) |
| created_at | author_id | body | |
| | created_at | created_at | |

Kluczowym elementem modelu jest ograniczenie na poziomie bazy danych: jeden użytkownik może oddać tylko jeden głos na dany element (wpis lub komentarz), co gwarantuje spójność wyników głosowania.

## 08 Dlaczego ten projekt

- Pełny cykl wytwarzania. Logowanie, formularze, operacje CRUD, API oraz baza danych — komplet elementów typowej aplikacji webowej.
- Ciekawe wyzwania techniczne. Budowa drzewa komentarzy oraz zapewnienie unikalności głosu na poziomie zapytań SQL.
- Łatwa rozbudowa. Zakres prac można elastycznie dopasować do dostępnego czasu.

### Kierunki rozwoju (na później)

Poza zakresem pierwszej wersji aplikacji pozostają pomysły na dalszy rozwój: system punktów XP i poziomów użytkowników, kategorie tematyczne wpisów oraz wyszukiwarka działająca po tagach.